paul graham

on making things

essays, volume four

contents

01	taste for makers	february 2002	6
02	hackers and painters	may 2003	14
03	design and research	january 2003	24
04	six principles for making new things	february 2008	30
05	the other half of ″artists ship″	november 2008	34
06	the hardware renaissance		38
07	crazy new ideas	may 2021	40
08	heresy	april 2022	44
09	what i′ve learned from users	september 2022	50
10	superlinear returns	october 2023	56
11	how art can be good	december 2006	64
12	how to do what you love		72

"...copernicus' aesthetic objections to [equants] provided one essential motive for his rejection of the ptolemaic system...."

- thomas kuhn, *the copernican revolution*

"all of us had been trained by kelly johnson and believed fanatically in his insistence that an airplane that looked beautiful would fly the same way."

- ben rich, *skunk works*

"beauty is the first test: there is no permanent place in this world for ugly mathematics."

- g. h. hardy, *a mathematician's apology*

i was talking recently to a friend who teaches at mit. his field is hot now and every year he is inundated by applications from would-be graduate students. "a lot of them seem smart," he said. "what i can't tell is whether they have any kind of taste."

taste. you don't hear that word much now. and yet we still need the underlying concept, whatever we call it. what my friend meant was that he wanted students who were not just good technicians, but who could use their technical knowledge to design beautiful things.

mathematicians call good work "beautiful," and so, either now or in the past, have scientists, engineers, musicians, architects, designers, writers, and painters. is it just a coincidence that they used the same word, or is there some overlap in what they meant? if there is an overlap, can we use one field's discoveries about beauty to help us in another?

for those of us who design things, these are not just theoretical questions. if there is such a thing as beauty, we need to be able to recognize it. we need good taste to make good things. instead of treating beauty as an airy abstraction, to be either blathered about or avoided depending on how one feels about airy abstractions, let's try considering it as a practical question: *how do you make good stuff?*

if you mention taste nowadays, a lot of people will tell you that "taste is subjective." they believe this because it really feels that way to them. when they like something, they have no idea why. it could be because it's beautiful, or because their mother had one, or because they saw a movie star with one in a magazine, or because they know it's expensive. their thoughts are a tangle of unexamined impulses.

most of us are encouraged, as children, to leave this tangle unexamined. if you make fun of your little brother for coloring people green in his coloring book, your mother is likely to tell you something like "you like to do it your way and he likes to do it his way."

your mother at this point is not trying to teach you important truths about aesthetics. she's trying to get the two of you to stop bickering.

like many of the half-truths adults tell us, this one contradicts other things they tell us. after dinning into you that taste is merely a matter of personal preference, they take you to the museum and tell you that you should pay attention because leonardo is a great artist.

what goes through the kid's head at this point? what does he think "great artist" means? after having been told for years that everyone just likes to do things their own way, he is unlikely to head straight for the conclusion that a great artist is someone whose work is *better* than the others'. a far more likely theory, in his ptolemaic model of the universe, is that a great artist is something that's good for you, like broccoli, because someone said so in a book.

must derive more from merit and less from fashion.

strangely enough, if you want to make something that will appeal to future generations, one way to do it is to try to appeal to past generations. it's hard to guess what the future will be like, but we can be sure it will be like the past in caring nothing for present fashions. so if you can make something that appeals to people today and would also have appealed to people in 1500, there is a good chance it will appeal to people in 2500.

good design solves the right problem. the typical stove has four burners arranged in a square, and a dial to control each. how do you arrange the dials? the simplest answer is to put them in a row. but this is a simple answer to the wrong question. the dials are for humans to use, and if you put them in a row, the unlucky human will have to stop and think each time about which dial matches which burner. better to arrange the dials in a square like the burners.

a lot of bad design is industrious, but misguided. in the mid twentieth century there was a vogue for setting text in sans-serif fonts. these fonts *are* closer to the pure, underlying letterforms. but in text that's not the problem you're trying to solve. for legibility it's more important that letters be easy to tell apart. it may look victorian, but a times roman lowercase g is easy to tell from a lowercase y.

problems can be improved as well as solutions. in software, an intractable problem can usually be replaced by an equivalent one that's easy to solve. physics progressed faster as the problem became predicting observable behavior, instead of reconciling it with scripture.

good design is suggestive. jane austen's novels contain almost no description; instead of telling you how everything looks, she tells her story so well that you envision the scene for yourself. likewise, a painting that suggests is usually more engaging than one that tells. everyone makes up their own story about the mona lisa.

in architecture and design, this principle means that a building or object should let you use it how you want: a good building, for example, will serve as a backdrop for whatever life people want to lead in it, instead of making them live as if they were executing a program written by the architect.

in software, it means you should give users a few basic elements that they can combine as they wish, like lego. in math it means a proof that becomes the basis for a lot of new work is preferable to a proof that was difficult, but doesn't lead to future discoveries; in the sciences generally, citation is considered a rough indicator of merit.

good design is often slightly funny. this one may not always be true. but durer's engravings and saarinen's womb chair and the pantheon and the original porsche 911 all seem to me slightly funny. godel's incompleteness theorem seems like a practical joke.

i think it's because humor is related to strength. to have a sense of humor is to be strong: to keep one's sense of humor is to shrug off misfortunes, and to lose one's sense of humor is to be wounded by them. and so the mark-- or at least the prerogative-- of strength is not to take oneself too seriously. the confident will often, like swallows, seem to be making fun of the whole process slightly, as hitchcock does in his films or bruegel in his paintings-- or shakespeare, for that matter.

good design may not have to be funny, but it's hard to imagine something that could be called humorless also being good design.

good design is hard. if you look at the people who've done great work, one thing they all seem to have in common is that they worked very hard. if you're not working hard, you're probably wasting your time.

symmetry is unfashionable in some fields now, in reaction to excesses in the past. architects started consciously making buildings asymmetric in victorian times and by the 1920s asymmetry was an explicit premise of modernist architecture. even these buildings only tended to be asymmetric about major axes, though; there were hundreds of minor symmetries.

in writing you find symmetry at every level, from the phrases in a sentence to the plot of a novel. you find the same in music and art. mosaics (and some cezannes) get extra visual punch by making the whole picture out of the same atoms. compositional symmetry yields some of the most memorable paintings, especially when two halves react to one another, as in the *creation of adam* or *american gothic*.

in math and engineering, recursion, especially, is a big win. inductive proofs are wonderfully short. in software, a problem that can be solved by recursion is nearly always best solved that way. the eiffel tower looks striking partly because it is a recursive solution, a tower on a tower.

the danger of symmetry, and repetition especially, is that it can be used as a substitute for thought.

good design resembles nature. it's not so much that resembling nature is intrinsically good as that nature has had a long time to work on the problem. it's a good sign when your answer resembles nature's.

it's not cheating to copy. few would deny that a story should be like life. working from life is a valuable tool in painting too, though its role has often been misunderstood. the aim is not simply to make a record. the point of painting from life is that it gives your mind something to chew on: when your eyes are looking at something, your hand will do more interesting work.

imitating nature also works in engineering. boats have long had spines and ribs like an animal's ribcage. in some cases we may have to wait for better technology: early aircraft designers were mistaken to design aircraft that looked like birds, because they didn't have materials or power sources light enough (the wrights' engine weighed 152 lbs. and generated only 12 hp.) or control systems sophisticated enough for machines that flew like birds, but i could imagine little unmanned reconnaissance planes flying like birds in fifty years.

now that we have enough computer power, we can imitate nature's method as well as its results. genetic algorithms may let us create things too complex to design in the ordinary sense.

good design is redesign. it's rare to get things right the first time. experts expect to throw away some early work. they plan for plans to change.

it takes confidence to throw work away. you have to be able to think, *there's more where that came from.* when people first start drawing, for example, they're often reluctant to redo parts that aren't right; they feel they've been lucky to get that far, and if they try to redo something, it will turn out worse. instead they convince themselves that the drawing is not that bad, really-- in fact, maybe they meant it to look that way.

dangerous territory, that; if anything you should cultivate dissatisfaction. in leonardo's drawings there are often five or six attempts to get a line right. the distinctive back of the porsche 911 only appeared in the redesign of an awkward prototype. in wright's early plans for the guggenheim, the right half was a ziggurat; he inverted it to get the present shape.

mistakes are natural. instead of treating them as disasters, make them easy to acknowledge and easy to fix. leonardo more or less invented the sketch, as a way to make drawing bear a greater weight of exploration. open-source software has fewer bugs because it admits the possibility of bugs.

we should be greeted daily by artistic marvels. we aren't, and the reason is that to make leonardo you need more than his innate ability. you also need florence in 1450.

nothing is more powerful than a community of talented people working on related problems. genes count for little by comparison: being a genetic leonardo was not enough to compensate for having been born near milan instead of florence. today we move around more, but great work still comes disproportionately from a few hotspots: the bauhaus, the manhattan project, the *new yorker,* lockheed's skunk works, xerox parc.

at any given time there are a few hot topics and a few groups doing great work on them, and it's nearly impossible to do good work yourself if you're too far removed from one of these centers. you can push or pull these trends to some extent, but you can't break away from them. (maybe *you* can, but the milanese leonardo couldn't.)

good design is often daring. at every period of history, people have believed things that were just ridiculous, and believed them so strongly that you risked ostracism or even violence by saying otherwise.

if our own time were any different, that would be remarkable. as far as i can tell it isn't.

this problem afflicts not just every era, but in some degree every field. much renaissance art was in its time considered shockingly secular: according to vasari, botticelli repented and gave up painting, and fra bartolommeo and lorenzo di credi actually burned some of their work. einstein's theory of relativity offended many contemporary physicists, and was not fully accepted for decades-- in france, not until the 1950s.

today's experimental error is tomorrow's new theory. if you want to discover great new things, then instead of turning a blind eye to the places where conventional wisdom and truth don't quite meet, you should pay particular attention to them.

as a practical matter, i think it's easier to see ugliness than to imagine beauty. most of the people who've made beautiful things seem to have done it by fixing something that they thought ugly. great work usually seems to happen because someone sees something and thinks, *i could do better than that*. giotto saw traditional byzantine madonnas painted according to a formula that had satisfied everyone for centuries, and to him they looked wooden and unnatural. copernicus was so troubled by a hack that all his contemporaries could tolerate that he felt there must be a better solution.

intolerance for ugliness is not in itself enough. you have to understand a field well before you develop a good nose for what needs fixing. you have to do your homework. but as you become expert in a field, you'll start to hear little voices saying, *what a hack! there must be a better way.* don't ignore those voices. cultivate them. the recipe for great work is: very exacting taste, plus the ability to gratify it.

---

notes

sullivan actually said "form ever follows function," but i think the usual misquotation is closer to what modernist architects meant.

stephen g. brush, "why was relativity accepted?" *phys. perspect. 1 (1999) 184-214.*

(this essay is derived from a guest lecture at harvard, which incorporated an earlier talk at northeastern.)

when i finished grad school in computer science i went to art school to study painting. a lot of people seemed surprised that someone interested in computers would also be interested in painting. they seemed to think that hacking and painting were very different kinds of work-- that hacking was cold, precise, and methodical, and that painting was the frenzied expression of some primal urge.

both of these images are wrong. hacking and painting have a lot in common. in fact, of all the different types of people i've known, hackers and painters are among the most alike.

what hackers and painters have in common is that they're both makers. along with composers, architects, and writers, what hackers and painters are trying to do is make good things. they're not doing research per se, though if in the course of trying to make good things they discover some new technique, so much the better.

i've never liked the term "computer science." the main reason i don't like it is that there's no such thing. computer science is a grab bag of tenuously related areas thrown together by an accident of history, like yugoslavia. at one end you have people who are really mathematicians, but call what they're doing computer science so they can get darpa grants. in the middle you have people working on something like the natural history of computers-- studying the behavior of algorithms for routing data through networks, for example. and then at the other extreme you have the hackers, who are trying to write interesting software, and for whom computers are just a medium of expression, as concrete is for architects or paint for painters. it's as if mathematicians, physicists, and architects all had to be in the same department.

sometimes what the hackers do is called "software engineering," but this term is just as misleading. good software designers are no more engineers than architects are. the border between architecture and engineering is not sharply defined, but it's there. it falls between what and how: architects decide what to do, and engineers figure out how to do it.

what and how should not be kept too separate. you're asking for trouble if you try to decide what to do without understanding how to do it. but hacking can certainly be more than just deciding how to implement some spec. at its best, it's creating the spec-- though it turns out the best way to do that is to implement it.

perhaps one day "computer science" will, like yugoslavia, get broken up into its component parts. that might be a good thing. especially if it meant independence for my native land, hacking.

bundling all these different types of work together in one department may be convenient administratively, but it's confusing intellectually. that's the other reason i don't like the name "computer science." arguably the people in the middle are doing something like an experimental science. but the people at either end, the hackers and the mathematicians, are not actually doing science.

the mathematicians don't seem bothered by this. they happily set to work proving theorems like the other mathematicians over in the math department, and probably soon stop noticing that the building they work in says ``computer science'' on the outside. but for the hackers this label is a problem. if what they're doing is called science, it makes them feel they ought to be acting scientific. so instead of doing what they really want to do, which is to design beautiful software, hackers in universities and research labs feel they ought to be writing research papers.

in the best case, the papers are just a formality. hackers write cool software, and then write a paper about it, and the paper becomes a proxy for the achievement represented by the software. but often

the theory of computation.

for example, i was taught in college that one ought to figure out a program completely on paper before even going near a computer. i found that i did not program this way. i found that i liked to program sitting in front of a computer, not a piece of paper. worse still, instead of patiently writing out a complete program and assuring myself it was correct, i tended to just spew out code that was hopelessly broken, and gradually beat it into shape. debugging, i was taught, was a kind of final pass where you caught typos and oversights. the way i worked, it seemed like programming consisted of debugging.

for a long time i felt bad about this, just as i once felt bad that i didn't hold my pencil the way they taught me to in elementary school. if i had only looked over at the other makers, the painters or the architects, i would have realized that there was a name for what i was doing: sketching. as far as i can tell, the way they taught me to program in college was all wrong. you should figure out programs as you're writing them, just as writers and painters and architects do.

realizing this has real implications for software design. it means that a programming language should, above all, be malleable. a programming language is for thinking of programs, not for expressing programs you've already thought of. it should be a pencil, not a pen. static typing would be a fine idea if people actually did write programs the way they taught me to in college. but that's not how any of the hackers i know write programs. we need a language that lets us scribble and smudge and smear, not a language where you have to sit with a teacup of types balanced on your knee and make polite conversation with a strict old aunt of a compiler.

while we're on the subject of static typing, identifying with the makers will save us from another problem that afflicts the sciences: math envy. everyone in the sciences secretly believes that mathematicians are smarter than they are. i think mathematicians also believe this. at any rate, the result is that scientists tend to make their work look as mathematical as possible. in a field like physics this probably doesn't do much harm, but the further you get from the natural sciences, the more of a problem it becomes.

a page of formulas just looks so impressive. (tip: for extra impressiveness, use greek variables.) and so there is a great temptation to work on problems you can treat formally, rather than problems that are, say, important.

if hackers identified with other makers, like writers and painters, they wouldn't feel tempted to do this. writers and painters don't suffer from math envy. they feel as if they're doing something completely unrelated. so are hackers, i think.

if universities and research labs keep hackers from doing the kind of work they want to do, perhaps the place for them is in companies. unfortunately, most companies won't let hackers do what they want either. universities and research labs force hackers to be scientists, and companies force them to be engineers.

i only discovered this myself quite recently. when yahoo bought viaweb, they asked me what i wanted to do. i had never liked the business side very much, and said that i just wanted to hack. when i got to yahoo, i found that what hacking meant to them was implementing software, not designing it. programmers were seen as technicians who translated the visions (if that is the word) of product managers into code.

this seems to be the default plan in big companies. they do it because it decreases the standard deviation of the outcome. only a small percentage of hackers can actually design software, and it's hard for the people running a company to pick these out. so instead of entrusting the future of the software to

that open-source is probably the right model, because it has been independently confirmed by all the other makers.

it seems surprising to me that any employer would be reluctant to let hackers work on open-source projects. at viaweb, we would have been reluctant to hire anyone who didn't. when we interviewed programmers, the main thing we cared about was what kind of software they wrote in their spare time. you can't do anything really well unless you love it, and if you love to hack you'll inevitably be working on projects of your own.[2]

because hackers are makers rather than scientists, the right place to look for metaphors is not in the sciences, but among other kinds of makers. what else can painting teach us about hacking?

one thing we can learn, or at least confirm, from the example of painting is how to learn to hack. you learn to paint mostly by doing it. ditto for hacking. most hackers don't learn to hack by taking college courses in programming. they learn to hack by writing programs of their own at age thirteen. even in college classes, you learn to hack mostly by hacking.[3]

because painters leave a trail of work behind them, you can watch them learn by doing. if you look at the work of a painter in chronological order, you'll find that each painting builds on things that have been learned in previous ones. when there's something in a painting that works very well, you can usually find version 1 of it in a smaller form in some earlier painting.

i think most makers work this way. writers and architects seem to as well. maybe it would be good for hackers to act more like painters, and regularly start over from scratch, instead of continuing to work for years on one project, and trying to incorporate all their later ideas as revisions.

the fact that hackers learn to hack by doing it is another sign of how different hacking is from the sciences. scientists don't learn science by doing it, but by doing labs and problem sets. scientists start out doing work that's perfect, in the sense that they're just trying to reproduce work someone else has already done for them. eventually, they get to the point where they can do original work. whereas hackers, from the start, are doing original work; it's just very bad. so hackers start original, and get good, and scientists start good, and get original.

the other way makers learn is from examples. for a painter, a museum is a reference library of techniques. for hundreds of years it has been part of the traditional education of painters to copy the works of the great masters, because copying forces you to look closely at the way a painting is made.

writers do this too. benjamin franklin learned to write by summarizing the points in the essays of addison and steele and then trying to reproduce them. raymond chandler did the same thing with detective stories.

hackers, likewise, can learn to program by looking at good programs-- not just at what they do, but the source code too. one of the less publicized benefits of the open-source movement is that it has made it easier to learn to program. when i learned to program, we had to rely mostly on examples in books. the one big chunk of code available then was unix, but even this was not open source. most of the people who read the source read it in illicit photocopies of john lions' book, which though written in 1977 was not allowed to be published until 1996.

another example we can take from painting is the way that paintings are created by gradual refinement. paintings usually begin with a sketch. gradually the details get filled in. but it is not merely a process of filling in. sometimes the original plans turn out to be mistaken. countless paintings, when

solve it. your program is supposed to do x. instead it does y. where does it go wrong? you know you're going to win in the end. it's as relaxing as painting a wall.

the example of painting can teach us not only how to manage our own work, but how to work together. a lot of the great art of the past is the work of multiple hands, though there may only be one name on the wall next to it in the museum. leonardo was an apprentice in the workshop of verrocchio and painted one of the angels in his baptism of christ. this sort of thing was the rule, not the exception. michelangelo was considered especially dedicated for insisting on painting all the figures on the ceiling of the sistine chapel himself.

as far as i know, when painters worked together on a painting, they never worked on the same parts. it was common for the master to paint the principal figures and for assistants to paint the others and the background. but you never had one guy painting over the work of another.

i think this is the right model for collaboration in software too. don't push it too far. when a piece of code is being hacked by three or four different people, no one of whom really owns it, it will end up being like a common-room. it will tend to feel bleak and abandoned, and accumulate cruft. the right way to collaborate, i think, is to divide projects into sharply defined modules, each with a definite owner, and with interfaces between them that are as carefully designed and, if possible, as articulated as programming languages.

like painting, most software is intended for a human audience. and so hackers, like painters, must have empathy to do really great work. you have to be able to see things from the user's point of view.

when i was a kid i was always being told to look at things from someone else's point of view. what this always meant in practice was to do what someone else wanted, instead of what i wanted. this of course gave empathy a bad name, and i made a point of not cultivating it.

boy, was i wrong. it turns out that looking at things from other people's point of view is practically the secret of success. it doesn't necessarily mean being self-sacrificing. far from it. understanding how someone else sees things doesn't imply that you'll act in his interest; in some situations-- in war, for example-- you want to do exactly the opposite.[4]

most makers make things for a human audience. and to engage an audience you have to understand what they need. nearly all the greatest paintings are paintings of people, for example, because people are what people are interested in.

empathy is probably the single most important difference between a good hacker and a great one. some hackers are quite smart, but when it comes to empathy are practically solipsists. it's hard for such people to design great software[5], because they can't see things from the user's point of view.

one way to tell how good people are at empathy is to watch them explain a technical question to someone without a technical background. we probably all know people who, though otherwise smart, are just comically bad at this. if someone asks them at a dinner party what a programming language is, they'll say something like ``oh, a high-level language is what the compiler uses as input to generate object code." high-level language? compiler? object code? someone who doesn't know what a programming language is obviously doesn't know what these things are, either.

part of what software has to do is explain itself. so to write good software you have to understand how little users understand. they're going to walk up to the software with no preparation, and it had better do what they guess it will, because they're not going to read the manual. the best system i've ever seen

[1] the greatest damage that photography has done to painting may be the fact that it killed the best day job. most of the great painters in history supported themselves by painting portraits.

[2] i've been told that microsoft discourages employees from contributing to open-source projects, even in their spare time. but so many of the best hackers work on open-source projects now that the main effect of this policy may be to ensure that they won't be able to hire any first-rate programmers.

[3] what you learn about programming in college is much like what you learn about books or clothes or dating: what bad taste you had in high school.

[4] here's an example of applied empathy. at viaweb, if we couldn't decide between two alternatives, we'd ask, what would our competitors hate most? at one point a competitor added a feature to their software that was basically useless, but since it was one of few they had that we didn't, they made much of it in the trade press. we could have tried to explain that the feature was useless, but we decided it would annoy our competitor more if we just implemented it ourselves, so we hacked together our own version that afternoon.

[5] except text editors and compilers. hackers don't need empathy to design these, because they are themselves typical users.

[6] well, almost. they overshot the available ram somewhat, causing much inconvenient disk swapping, but this could be fixed within a few months by buying an additional disk drive.

[7] the way to make programs easy to read is not to stuff them with comments. i would take abelson and sussman's quote a step further. programming languages should be designed to express algorithms, and only incidentally to tell computers how to execute them. a good programming language ought to be better for explaining software than english. you should only need comments when there is some kind of kludge you need to warn readers about, just as on a road there are only arrows on parts with unexpectedly sharp curves.

thanks to trevor blackwell, robert morris, dan giffin, and lisa randall for reading drafts of this, and to henry leitner and larry finkelstein for inviting me to speak.

(this article is derived from a keynote talk at the fall 2002 meeting of nepls.)

visitors to this country are often surprised to find that americans like to begin a conversation by asking "what do you do?" i've never liked this question. i've rarely had a neat answer to it. but i think i have finally solved the problem. now, when someone asks me what i do, i look them straight in the eye and say "i'm designing a new dialect of lisp." i recommend this answer to anyone who doesn't like being asked what they do. the conversation will turn immediately to other topics.

i don't consider myself to be doing research on programming languages. i'm just designing one, in the same way that someone might design a building or a chair or a new typeface. i'm not trying to discover anything new. i just want to make a language that will be good to program in. in some ways, this assumption makes life a lot easier.

the difference between design and research seems to be a question of new versus good. design doesn't have to be new, but it has to be good. research doesn't have to be good, but it has to be new. i think these two paths converge at the top: the best design surpasses its predecessors by using new ideas, and the best research solves problems that are not only new, but actually worth solving. so ultimately we're aiming for the same destination, just approaching it from different directions.

what i'm going to talk about today is what your target looks like from the back. what do you do differently when you treat programming languages as a design problem instead of a research topic?

the biggest difference is that you focus more on the user. design begins by asking, who is this for and what do they need from it? a good architect, for example, does not begin by creating a design that he then imposes on the users, but by studying the intended users and figuring out what they need.

notice i said "what they need," not "what they want." i don't mean to give the impression that working as a designer means working as a sort of short-order cook, making whatever the client tells you to. this varies from field to field in the arts, but i don't think there is any field in which the best work is done by the people who just make exactly what the customers tell them to.

the customer *is* always right in the sense that the measure of good design is how well it works for the user. if you make a novel that bores everyone, or a chair that's horribly uncomfortable to sit in, then you've done a bad job, period. it's no defense to say that the novel or the chair is designed according to the most advanced theoretical principles.

and yet, making what works for the user doesn't mean simply making what the user tells you to. users don't know what all the choices are, and are often mistaken about what they really want.

the answer to the paradox, i think, is that you have to design for the user, but you have to design what the user needs, not simply what he says he wants. it's much like being a doctor. you can't just treat a patient's symptoms. when a patient tells you his symptoms, you have to figure out what's actually wrong with him, and treat that.

this focus on the user is a kind of axiom from which most of the practice of good design can be derived, and around which most design issues center.

if good design must do what the user needs, who is the user? when i say that design must be for users, i don't mean to imply that good design aims at some kind of lowest common denominator. you can pick any group of users you want. if you're designing a tool, for example, you can design it for anyone from beginners to experts, and what's good design for one group might be bad for another. the point is, you have to pick some group of users. i don't think you can even talk about good or bad design except with reference to some intended user.

dispute either of the premises, but if a language has to have a toplevel to be convenient, and mandatory type declarations are incompatible with a toplevel, then no language that makes type declarations mandatory could be convenient to program in.

in practice, to get good design you have to get close, and stay close, to your users. you have to calibrate your ideas on actual users constantly, especially in the beginning. one of the reasons jane austen's novels are so good is that she read them out loud to her family. that's why she never sinks into self-indulgently arty descriptions of landscapes, or pretentious philosophizing. (the philosophy's there, but it's woven into the story instead of being pasted onto it like a label.) if you open an average "literary" novel and imagine reading it out loud to your friends as something you'd written, you'll feel all too keenly what an imposition that kind of thing is upon the reader.

in the software world, this idea is known as worse is better. actually, there are several ideas mixed together in the concept of worse is better, which is why people are still arguing about whether worse is actually better or not. but one of the main ideas in that mix is that if you're building something new, you should get a prototype in front of users as soon as possible.

the alternative approach might be called the hail mary strategy. instead of getting a prototype out quickly and gradually refining it, you try to create the complete, finished, product in one long touchdown pass. as far as i know, this is a recipe for disaster. countless startups destroyed themselves this way during the internet bubble. i've never heard of a case where it worked.

what people outside the software world may not realize is that worse is better is found throughout the arts. in drawing, for example, the idea was discovered during the renaissance. now almost every drawing teacher will tell you that the right way to get an accurate drawing is not to work your way slowly around the contour of an object, because errors will accumulate and you'll find at the end that the lines don't meet. instead you should draw a few quick lines in roughly the right place, and then gradually refine this initial sketch.

in most fields, prototypes have traditionally been made out of different materials. typefaces to be cut in metal were initially designed with a brush on paper. statues to be cast in bronze were modelled in wax. patterns to be embroidered on tapestries were drawn on paper with ink wash. buildings to be constructed from stone were tested on a smaller scale in wood.

what made oil paint so exciting, when it first became popular in the fifteenth century, was that you could actually make the finished work *from* the prototype. you could make a preliminary drawing if you wanted to, but you weren't held to it; you could work out all the details, and even make major changes, as you finished the painting.

you can do this in software too. a prototype doesn't have to be just a model; you can refine it into the finished product. i think you should always do this when you can. it lets you take advantage of new insights you have along the way. but perhaps even more important, it's good for morale.

morale is key in design. i'm surprised people don't talk more about it. one of my first drawing teachers told me: if you're bored when you're drawing something, the drawing will look boring. for example, suppose you have to draw a building, and you decide to draw each brick individually. you can do this if you want, but if you get bored halfway through and start making the bricks mechanically instead of observing each one, the drawing will look worse than if you had merely suggested the bricks.

building something by gradually refining a prototype is good for morale because it keeps you engaged. in software, my rule is: always have working code. if you're writing something that you'll be able to test in an hour, then you have the prospect of an immediate reward to motivate you. the same is true in the

the fiery reaction to the release of arc had an unexpected consequence: it made me realize i had a design philosophy. the main complaint of the more articulate critics was that arc seemed so flimsy. after years of working on it, all i had to show for myself were a few thousand lines of macros? why hadn't i worked on more substantial problems?

as i was mulling over these remarks it struck me how familiar they seemed. this was exactly the kind of thing people said at first about viaweb, and y combinator, and most of my essays.

when we launched viaweb, it seemed laughable to vcs and e-commerce "experts." we were just a couple guys in an apartment, which did not seem cool in 1995 the way it does now. and the thing we'd built, as far as they could tell, wasn't even software. software, to them, equalled big, honking windows apps. since viaweb was the first web-based app they'd seen, it seemed to be nothing more than a website. they were even more contemptuous when they discovered that viaweb didn't process credit card transactions (we didn't for the whole first year). transaction processing seemed to them what e-commerce was all about. it sounded serious and difficult.

and yet, mysteriously, viaweb ended up crushing all its competitors.

the initial reaction to y combinator was almost identical. it seemed laughably lightweight. startup funding meant series a rounds: millions of dollars given to a small number of startups founded by people with established credentials after months of serious, businesslike meetings, on terms described in a document a foot thick. y combinator seemed inconsequential. it's too early to say yet whether y combinator will turn out like viaweb, but judging from the number of imitations, a lot of people seem to think we're on to something.

i can't measure whether my essays are successful, except in page views, but the reaction to them is at least different from when i started. at first the default reaction of the slashdot trolls was (translated into articulate terms): "who is this guy and what authority does he have to write about these topics? i haven't read the essay, but there's no way anything so short and written in such an informal style could have anything useful to say about such and such topic, when people with degrees in the subject have already written many thick books about it." now there's a new generation of trolls on a new generation of sites, but they have at least started to omit the initial "who is this guy?"

now people are saying the same things about arc that they said at first about viaweb and y combinator and most of my essays. why the pattern? the answer, i realized, is that my m.o. for all four has been the same.

here it is: i like to find (a) simple solutions (b) to overlooked problems (c) that actually need to be solved, and (d) deliver them as informally as possible, (e) starting with a very crude version 1, then (f) iterating rapidly.

when i first laid out these principles explicitly, i noticed something striking: this is practically a recipe for generating a contemptuous initial reaction. though simple solutions are better, they don't seem as impressive as complex ones. overlooked problems are by definition problems that most people think don't matter. delivering solutions in an informal way means that instead of judging something by the way it's presented, people have to actually understand it, which is more work. and starting with a crude version 1 means your initial effort is always small and incomplete.

i'd noticed, of course, that people never seemed to grasp new ideas at first. i thought it was just because most people were stupid. now i see there's more to it than that. like a contrarian investment fund, someone following this strategy will almost always be doing things that seem wrong to the average person.

one of the differences between big companies and startups is that big companies tend to have developed procedures to protect themselves against mistakes. a startup walks like a toddler, bashing into things and falling over all the time. a big company is more deliberate.

the gradual accumulation of checks in an organization is a kind of learning, based on disasters that have happened to it or others like it. after giving a contract to a supplier who goes bankrupt and fails to deliver, for example, a company might require all suppliers to prove they're solvent before submitting bids.

as companies grow they invariably get more such checks, either in response to disasters they've suffered, or (probably more often) by hiring people from bigger companies who bring with them customs for protecting against new types of disasters.

it's natural for organizations to learn from mistakes. the problem is, people who propose new checks almost never consider that the check itself has a cost.

every check has a cost. for example, consider the case of making suppliers verify their solvency. surely that's mere prudence? but in fact it could have substantial costs. there's obviously the direct cost in time of the people on both sides who supply and check proofs of the supplier's solvency. but the real costs are the ones you never hear about: the company that would be the best supplier, but doesn't bid because they can't spare the effort to get verified. or the company that would be the best supplier, but falls just short of the threshold for solvency—which will of course have been set on the high side, since there is no apparent cost of increasing it.

whenever someone in an organization proposes to add a new check, they should have to explain not just the benefit but the cost. no matter how bad a job they did of analyzing it, this meta-check would at least remind everyone there had to *be* a cost, and send them looking for it.

if companies started doing that, they'd find some surprises. joel spolsky recently spoke at y combinator about selling software to corporate customers. he said that in most companies software costing up to about $1000 could be bought by individual managers without any additional approvals. above that threshold, software purchases generally had to be approved by a committee. but babysitting this process was so expensive for software vendors that it didn't make sense to charge less than $50,000. which means if you're making something you might otherwise have charged $5000 for, you have to sell it for $50,000 instead.

the purpose of the committee is presumably to ensure that the company doesn't waste money. and yet the result is that the company pays 10 times as much.

checks on purchases will always be expensive, because the harder it is to sell something to you, the more it has to cost. and not merely linearly, either. if you're hard enough to sell to, the people who are best at making things don't want to bother. the only people who will sell to you are companies that specialize in selling to you. then you've sunk to a whole new level of inefficiency. market mechanisms no longer protect you, because the good suppliers are no longer in the market.

such things happen constantly to the biggest organizations of all, governments. but checks instituted by governments can cause much worse problems than merely overpaying. checks instituted by governments can cripple a country's whole economy. up till about 1400, china was richer and more technologically advanced than europe. one reason europe pulled ahead was that the chinese government restricted long trading voyages. so it was left to the europeans to explore and eventually to dominate the rest of the world, including china.

october 2012

one advantage of y combinator's early, broad focus is that we see trends before most other people. and one of the most conspicuous trends in the last batch was the large number of hardware startups. out of 84 companies, 7 were making hardware. on the whole they've done better than the companies that weren't.

they've faced resistance from investors of course. investors have a deep-seated bias against hardware. but investors' opinions are a trailing indicator. the best founders are better at seeing the future than the best investors, because the best founders are making it.

there is no one single force driving this trend. hardware does well on crowdfunding sites. the spread of tablets makes it possible to build new things controlled by and even incorporating them. electric motors have improved. wireless connectivity of various types can now be taken for granted. it's getting more straightforward to get things manufactured. arduinos, 3d printing, laser cutters, and more accessible cnc milling are making hardware easier to prototype. retailers are less of a bottleneck as customers increasingly buy online.

one question i can answer is why hardware is suddenly cool. it always was cool. physical things are great. they just haven't been as great a way to start a rapidly growing business as software. but that rule may not be permanent. it's not even that old; it only dates from about 1990. maybe the advantage of software will turn out to have been temporary. hackers love to build hardware, and customers love to buy it. so if the ease of shipping hardware even approached the ease of shipping software, we'd see a lot more hardware startups.

it wouldn't be the first time something was a bad idea till it wasn't. and it wouldn't be the first time investors learned that lesson from founders.

so if you want to work on hardware, don't be deterred from doing it because you worry investors will discriminate against you. and in particular, don't be deterred from applying to y combinator with a hardware idea, because we're especially interested in hardware startups.

we know there's room for the next steve jobs. but there's almost certainly also room for the first <your name here>.

thanks to sam altman, trevor blackwell, david cann, sanjay dastoor, paul gerhardt, cameron robertson, harj taggar, and garry tan for reading drafts of this.

there's one kind of opinion i'd be very afraid to express publicly. if someone i knew to be both a domain expert and a reasonable person proposed an idea that sounded preposterous, i'd be very reluctant to say "that will never work."

anyone who has studied the history of ideas, and especially the history of science, knows that's how big things start. someone proposes an idea that sounds crazy, most people dismiss it, then it gradually takes over the world.

most implausible-sounding ideas are in fact bad and could be safely dismissed. but not when they're proposed by reasonable domain experts. if the person proposing the idea is reasonable, then they know how implausible it sounds. and yet they're proposing it anyway. that suggests they know something you don't. and if they have deep domain expertise, that's probably the source of it. [1]

such ideas are not merely unsafe to dismiss, but disproportionately likely to be interesting. when the average person proposes an implausible-sounding idea, its implausibility is evidence of their incompetence. but when a reasonable domain expert does it, the situation is reversed. there's something like an efficient market here: on average the ideas that seem craziest will, if correct, have the biggest effect. so if you can eliminate the theory that the person proposing an implausible-sounding idea is incompetent, its implausibility switches from evidence that it's boring to evidence that it's exciting. [2]

such ideas are not guaranteed to work. but they don't have to be. they just have to be sufficiently good bets — to have sufficiently high expected value. and i think on average they do. i think if you bet on the entire set of implausible-sounding ideas proposed by reasonable domain experts, you'd end up net ahead.

the reason is that everyone is too conservative. the word "paradigm" is overused, but this is a case where it's warranted. everyone is too much in the grip of the current paradigm. even the people who have the new ideas undervalue them initially. which means that before they reach the stage of proposing them publicly, they've already subjected them to an excessively strict filter. [3]

the wise response to such an idea is not to make statements, but to ask questions, because there's a real mystery here. why has this smart and reasonable person proposed an idea that seems so wrong? are they mistaken, or are you? one of you has to be. if you're the one who's mistaken, that would be good to know, because it means there's a hole in your model of the world. but even if they're mistaken, it should be interesting to learn why. a trap that an expert falls into is one you have to worry about too.

this all seems pretty obvious. and yet there are clearly a lot of people who don't share my fear of dismissing new ideas. why do they do it? why risk looking like a jerk now and a fool later, instead of just reserving judgement?

one reason they do it is envy. if you propose a radical new idea and it succeeds, your reputation (and perhaps also your wealth) will increase proportionally. some people would be envious if that happened, and this potential envy propagates back into a conviction that you must be wrong.

another reason people dismiss new ideas is that it's an easy way to seem sophisticated. when a new idea first emerges, it usually seems pretty feeble. it's a mere hatchling. received wisdom is a full-grown eagle by comparison. so it's easy to launch a devastating attack on a new idea, and anyone who does will seem clever to those who don't understand this asymmetry.

this phenomenon is exacerbated by the difference between how those working on new ideas and those attacking them are rewarded. the rewards for working on new ideas are weighted by the value of the outcome. so it's worth working on something that only has a 10% chance of succeeding if it would

5 hall, rupert. *from galileo to newton.* collins, 1963. this book is particularly good at getting into contemporaries' heads.

thanks to trevor blackwell, patrick collison, suhail doshi, daniel gackle, jessica livingston, and robert morris for reading drafts of this.

one of the most surprising things i've witnessed in my lifetime is the rebirth of the concept of heresy.

in his excellent biography of newton, richard westfall writes about the moment when he was elected a fellow of trinity college:

> supported comfortably, newton was free to devote himself wholly to whatever he chose. to remain on, he had only to avoid the three unforgivable sins: crime, heresy, and marriage. [1]

the first time i read that, in the 1990s, it sounded amusingly medieval. how strange, to have to avoid committing heresy. but when i reread it 20 years later it sounded like a description of contemporary employment.

there are an ever-increasing number of opinions you can be fired for. those doing the firing don't use the word "heresy" to describe them, but structurally they're equivalent. structurally there are two distinctive things about heresy: (1) that it takes priority over the question of truth or falsity, and (2) that it outweighs everything else the speaker has done.

for example, when someone calls a statement "x-ist," they're also implicitly saying that this is the end of the discussion. they do not, having said this, go on to consider whether the statement is true or not. using such labels is the conversational equivalent of signalling an exception. that's one of the reasons they're used: to end a discussion.

if you find yourself talking to someone who uses these labels a lot, it might be worthwhile to ask them explicitly if they believe any babies are being thrown out with the bathwater. can a statement be x-ist, for whatever value of x, and also true? if the answer is yes, then they're admitting to banning the truth. that's obvious enough that i'd guess most would answer no. but if they answer no, it's easy to show that they're mistaken, and that in practice such labels are applied to statements regardless of their truth or falsity.

the clearest evidence of this is that whether a statement is considered x-ist often depends on who said it. truth doesn't work that way. the same statement can't be true when one person says it, but x-ist, and therefore false, when another person does. [2]

the other distinctive thing about heresies, compared to ordinary opinions, is that the public expression of them outweighs everything else the speaker has done. in ordinary matters, like knowledge of history, or taste in music, you're judged by the average of your opinions. a heresy is qualitatively different. it's like dropping a chunk of uranium onto the scale.

back in the day (and still, in some places) the punishment for heresy was death. you could have led a life of exemplary goodness, but if you publicly doubted, say, the divinity of christ, you were going to burn. nowadays, in civilized countries, heretics only get fired in the metaphorical sense, by losing their jobs. but the structure of the situation is the same: the heresy outweighs everything else. you could have spent the last ten years saving children's lives, but if you express certain opinions, you're automatically fired.

it's much the same as if you committed a crime. no matter how virtuously you've lived, if you commit a crime, you must still suffer the penalty of the law. having lived a previously blameless life might mitigate the punishment, but it doesn't affect whether you're guilty or not.

a heresy is an opinion whose expression is treated like a crime — one that makes some people feel not merely that you're mistaken, but that you should be punished. indeed, their desire to see you punished is often stronger than it would be if you'd committed an actual crime. there are many on the far left

there are aggressively conventional-minded people on both the right and the left. the reason the current wave of intolerance comes from the left is simply because the new unifying ideology happened to come from the left. the next one might come from the right. imagine what that would be like.

fortunately in western countries the suppression of heresies is nothing like as bad as it used to be. though the window of opinions you can express publicly has narrowed in the last decade, it's still much wider than it was a few hundred years ago. the problem is the derivative. up till about 1985 the window had been growing ever wider. anyone looking into the future in 1985 would have expected freedom of expression to continue to increase. instead it has decreased. [5]

the situation is similar to what's happened with infectious diseases like measles. anyone looking into the future in 2010 would have expected the number of measles cases in the us to continue to decrease. instead, thanks to anti-vaxxers, it has increased. the absolute number is still not that high. the problem is the derivative. [6]

in both cases it's hard to know how much to worry. is it really dangerous to society as a whole if a handful of extremists refuse to get their kids vaccinated, or shout down speakers at universities? the point to start worrying is presumably when their efforts start to spill over into everyone else's lives. and in both cases that does seem to be happening.

so it's probably worth spending some amount of effort on pushing back to keep open the window of free expression. my hope is that this essay will help form social antibodies not just against current efforts to suppress ideas, but against the concept of heresy in general. that's the real prize. how do you disable the concept of heresy? since the enlightenment, western societies have discovered many techniques for doing that, but there are surely more to be discovered.

overall i'm optimistic. though the trend in freedom of expression has been bad over the last decade, it's been good over the longer term. and there are signs that the current wave of intolerance is peaking. independent-minded people i talk to seem more confident than they did a few years ago. on the other side, even some of the leaders are starting to wonder if things have gone too far. and popular culture among the young has already moved on. all we have to do is keep pushing back, and the wave collapses. and then we'll be net ahead, because as well as having defeated this wave, we'll also have developed new tactics for resisting the next one.

---

notes

1 or more accurately, biographies of newton, since westfall wrote two: a long version called *never at rest*, and a shorter one called *the life of isaac newton*. both are great. the short version moves faster, but the long one is full of interesting and often very funny details. this passage is the same in both.

2 another more subtle but equally damning bit of evidence is that claims of x-ism are never qualified. you never hear anyone say that a statement is "probably x-ist" or "almost certainly y-ist." if claims of x-ism were actually claims about truth, you'd expect to see "probably" in front of "x-ist" as often as you see it in front of "fallacious."

3 the rules must be strict, but they need not be demanding. so the most effective type of rules are those about superficial matters, like doctrinal minutiae, or the precise words adherents must use. such rules can be made extremely complicated, and yet don't repel potential converts by requiring significant sacrifice.

the superficial demands of orthodoxy make it an inexpensive substitute for virtue. and that in turn is one of the reasons orthodoxy is so attractive to bad people. you could be a horrible person, and yet as

i recently told applicants to y combinator that the best advice i could give for getting in, per word, was

> explain what you've learned from users.

that tests a lot of things: whether you're paying attention to users, how well you understand them, and even how much they need what you're making.

afterward i asked myself the same question. what have i learned from yc's users, the startups we've funded?

the first thing that came to mind was that most startups have the same problems. no two have exactly the same problems, but it's surprising how much the problems remain the same, regardless of what they're making. once you've advised 100 startups all doing different things, you rarely encounter problems you haven't seen before.

this fact is one of the things that makes yc work. but i didn't know it when we started yc. i only had a few data points: our own startup, and those started by friends. it was a surprise to me how often the same problems recur in different forms. many later stage investors might never realize this, because later stage investors might not advise 100 startups in their whole career, but a yc partner will get this much experience in the first year or two.

that's one advantage of funding large numbers of early stage companies rather than smaller numbers of later-stage ones. you get a lot of data. not just because you're looking at more companies, but also because more goes wrong.

but knowing (nearly) all the problems startups can encounter doesn't mean that advising them can be automated, or reduced to a formula. there's no substitute for individual office hours with a yc partner. each startup is unique, which means they have to be advised by specific partners who know them well. [1]

we learned that the hard way, in the notorious "batch that broke yc" in the summer of 2012. up till that point we treated the partners as a pool. when a startup requested office hours, they got the next available slot posted by any partner. that meant every partner had to know every startup. this worked fine up to 60 startups, but when the batch grew to 80, everything broke. the founders probably didn't realize anything was wrong, but the partners were confused and unhappy because halfway through the batch they still didn't know all the companies yet. [2]

at first i was puzzled. how could things be fine at 60 startups and broken at 80? it was only a third more. then i realized what had happened. we were using an $o(n^2)$ algorithm. so of course it blew up.

the solution we adopted was the classic one in these situations. we sharded the batch into smaller groups of startups, each overseen by a dedicated group of partners. that fixed the problem, and has worked fine ever since. but the batch that broke yc was a powerful demonstration of how individualized the process of advising startups has to be.

another related surprise is how bad founders can be at realizing what their problems are. founders will sometimes come in to talk about some problem, and we'll discover another much bigger one in the course of the conversation. for example (and this case is all too common), founders will come in to talk about the difficulties they're having raising money, and after digging into their situation, it turns out the reason is that the company is doing badly, and investors can tell. or founders will come in worried that they still haven't cracked the problem of user acquisition, and the reason turns out to be that their product isn't good enough. there have been times when i've asked "would you use this yourself, if you

because startups are so counterintuitive generally. and partly because many founders, especially young and ambitious ones, have been trained to win the wrong way. that took me years to figure out. the educational system in most countries trains you to win by hacking the test instead of actually doing whatever it's supposed to measure. but that stops working when you start a startup. so part of what yc does is to retrain founders to stop trying to hack the test. (it takes a surprisingly long time. a year in, you still see them reverting to their old habits.)

yc is not simply more experienced founders passing on their knowledge. it's more like specialization than apprenticeship. the knowledge of the yc partners and the founders have different shapes: it wouldn't be worthwhile for a founder to acquire the encyclopedic knowledge of startup problems that a yc partner has, just as it wouldn't be worthwhile for a yc partner to acquire the depth of domain knowledge that a founder has. that's why it can still be valuable for an experienced founder to do yc, just as it can still be valuable for an experienced athlete to have a coach.

the other big thing yc gives founders is colleagues, and this may be even more important than the advice of partners. if you look at history, great work clusters around certain places and institutions: florence in the late 15th century, the university of göttingen in the late 19th, *the new yorker* under ross, bell labs, xerox parc. however good you are, good colleagues make you better. indeed, very ambitious people probably need colleagues more than anyone else, because they're so starved for them in everyday life.

whether or not yc manages one day to be listed alongside those famous clusters, it won't be for lack of trying. we were very aware of this historical phenomenon and deliberately designed yc to be one. by this point it's not bragging to say that it's the biggest cluster of great startup founders. even people trying to attack yc concede that.

colleagues and startup founders are two of the most powerful forces in the world, so you'd expect it to have a big effect to combine them. before yc, to the extent people thought about the question at all, most assumed they couldn't be combined — that loneliness was the price of independence. that was how it felt to us when we started our own startup in boston in the 1990s. we had a handful of older people we could go to for advice (of varying quality), but no peers. there was no one we could commiserate with about the misbehavior of investors, or speculate with about the future of technology. i often tell founders to make something they themselves want, and yc is certainly that: it was designed to be exactly what we wanted when we were starting a startup.

one thing we wanted was to be able to get seed funding without having to make the rounds of random rich people. that has become a commodity now, at least in the us. but great colleagues can never become a commodity, because the fact that they cluster in some places means they're proportionally absent from the rest.

something magical happens where they do cluster though. the energy in the room at a yc dinner is like nothing else i've experienced. we would have been happy just to have one or two other startups to talk to. when you have a whole roomful it's another thing entirely.

yc founders aren't just inspired by one another. they also help one another. that's the happiest thing i've learned about startup founders: how generous they can be in helping one another. we noticed this in the first batch and consciously designed yc to magnify it. the result is something far more intense than, say, a university. between the partners, the alumni, and their batchmates, founders are surrounded by people who want to help them, and can.

one of the most important things i didn't understand about the world when i was a child is the degree to which the returns for performance are superlinear.

teachers and coaches implicitly told us the returns were linear. "you get out," i heard a thousand times, "what you put in." they meant well, but this is rarely true. if your product is only half as good as your competitor's, you don't get half as many customers. you get no customers, and you go out of business.

it's obviously true that the returns for performance are superlinear in business. some think this is a flaw of capitalism, and that if we changed the rules it would stop being true. but superlinear returns for performance are a feature of the world, not an artifact of rules we've invented. we see the same pattern in fame, power, military victories, knowledge, and even benefit to humanity. in all of these, the rich get richer. [1]

you can't understand the world without understanding the concept of superlinear returns. and if you're ambitious you definitely should, because this will be the wave you surf on.

it may seem as if there are a lot of different situations with superlinear returns, but as far as i can tell they reduce to two fundamental causes: exponential growth and thresholds.

the most obvious case of superlinear returns is when you're working on something that grows exponentially. for example, growing bacterial cultures. when they grow at all, they grow exponentially. but they're tricky to grow. which means the difference in outcome between someone who's adept at it and someone who's not is very great.

startups can also grow exponentially, and we see the same pattern there. some manage to achieve high growth rates. most don't. and as a result you get qualitatively different outcomes: the companies with high growth rates tend to become immensely valuable, while the ones with lower growth rates may not even survive.

y combinator encourages founders to focus on growth rate rather than absolute numbers. it prevents them from being discouraged early on, when the absolute numbers are still low. it also helps them decide what to focus on: you can use growth rate as a compass to tell you how to evolve the company. but the main advantage is that by focusing on growth rate you tend to get something that grows exponentially.

yc doesn't explicitly tell founders that with growth rate "you get out what you put in," but it's not far from the truth. and if growth rate were proportional to performance, then the reward for performance p over time t would be proportional to p^t.

even after decades of thinking about this, i find that sentence startling.

whenever how well you do depends on how well you've done, you'll get exponential growth. but neither our dna nor our customs prepare us for it. no one finds exponential growth natural; every child is surprised, the first time they hear it, by the story of the man who asks the king for a single grain of rice the first day and double the amount each successive day.

what we don't understand naturally we develop customs to deal with, but we don't have many customs about exponential growth either, because there have been so few instances of it in human history. in principle herding should have been one: the more animals you had, the more offspring they'd have. but in practice grazing land was the limiting factor, and there was no plan for growing that exponentially.

or more precisely, no generally applicable plan. there *was* a way to grow one's territory exponentially: by conquest. the more territory you control, the more powerful your army becomes, and the easier it is

in fact a good bet: maybe your company didn't grow the way you wanted, but you yourself have, and that should yield results eventually.

indeed, the forms of exponential growth that don't consist of learning are so often intermixed with it that we should probably treat this as the rule rather than the exception. which yields another heuristic: always be learning. if you're not learning, you're probably not on a path that leads to superlinear returns.

but don't overoptimize *what* you're learning. don't limit yourself to learning things that are already known to be valuable. you're learning; you don't know for sure yet what's going to be valuable, and if you're too strict you'll lop off the outliers.

what about step functions? are there also useful heuristics of the form "seek thresholds" or "seek competition?" here the situation is trickier. the existence of a threshold doesn't guarantee the game will be worth playing. if you play a round of russian roulette, you'll be in a situation with a threshold, certainly, but in the best case you're no better off. "seek competition" is similarly useless; what if the prize isn't worth competing for? sufficiently fast exponential growth guarantees both the shape and magnitude of the return curve — because something that grows fast enough will grow big even if it's trivially small at first — but thresholds only guarantee the shape. [4]

a principle for taking advantage of thresholds has to include a test to ensure the game is worth playing. here's one that does: if you come across something that's mediocre yet still popular, it could be a good idea to replace it. for example, if a company makes a product that people dislike yet still buy, then presumably they'd buy a better alternative if you made one. [5]

it would be great if there were a way to find promising intellectual thresholds. is there a way to tell which questions have whole new fields beyond them? i doubt we could ever predict this with certainty, but the prize is so valuable that it would be useful to have predictors that were even a little better than random, and there's hope of finding those. we can to some degree predict when a research problem *isn't* likely to lead to new discoveries: when it seems legit but boring. whereas the kind that do lead to new discoveries tend to seem very mystifying, but perhaps unimportant. (if they were mystifying and obviously important, they'd be famous open questions with lots of people already working on them.) so one heuristic here is to be driven by curiosity rather than careerism — to give free rein to your curiosity instead of working on what you're supposed to.

the prospect of superlinear returns for performance is an exciting one for the ambitious. and there's good news in this department: this territory is expanding in both directions. there are more types of work in which you can get superlinear returns, and the returns themselves are growing.

there are two reasons for this, though they're so closely intertwined that they're more like one and a half: progress in technology, and the decreasing importance of organizations.

fifty years ago it used to be much more necessary to be part of an organization to work on ambitious projects. it was the only way to get the resources you needed, the only way to have colleagues, and the only way to get distribution. so in 1970 your prestige was in most cases the prestige of the organization you belonged to. and prestige was an accurate predictor, because if you weren't part of an organization, you weren't likely to achieve much. there were a handful of exceptions, most notably artists and writers, who worked alone using inexpensive tools and had their own brands. but even they were at the mercy of organizations for reaching audiences. [6]

a world dominated by organizations damped variation in the returns for performance. but this world has eroded significantly just in my lifetime. now a lot more people can have the freedom that artists

and there is of course one other thing you need: to be lucky. luck is always a factor, but it's even more of a factor when you're working on your own rather than as part of an organization. and though there are some valid aphorisms about luck being where preparedness meets opportunity and so on, there's also a component of true chance that you can't do anything about. the solution is to take multiple shots. which is another reason to start taking risks early.

the best example of a field with superlinear returns is probably science. it has exponential growth, in the form of learning, combined with thresholds at the extreme edge of performance — literally at the limits of knowledge.

the result has been a level of inequality in scientific discovery that makes the wealth inequality of even the most stratified societies seem mild by comparison. newton's discoveries were arguably greater than all his contemporaries' combined. [11]

this point may seem obvious, but it might be just as well to spell it out. superlinear returns imply inequality. the steeper the return curve, the greater the variation in outcomes.

in fact, the correlation between superlinear returns and inequality is so strong that it yields another heuristic for finding work of this type: look for fields where a few big winners outperform everyone else. a kind of work where everyone does about the same is unlikely to be one with superlinear returns.

what are fields where a few big winners outperform everyone else? here are some obvious ones: sports, politics, art, music, acting, directing, writing, math, science, starting companies, and investing. in sports the phenomenon is due to externally imposed thresholds; you only need to be a few percent faster to win every race. in politics, power grows much as it did in the days of emperors. and in some of the other fields (including politics) success is driven largely by fame, which has its own source of superlinear growth. but when we exclude sports and politics and the effects of fame, a remarkable pattern emerges: the remaining list is exactly the same as the list of fields where you have to be independent-minded to succeed — where your ideas have to be not just correct, but novel as well. [12]

this is obviously the case in science. you can't publish papers saying things that other people have already said. but it's just as true in investing, for example. it's only useful to believe that a company will do well if most other investors don't; if everyone else thinks the company will do well, then its stock price will already reflect that, and there's no room to make money.

what else can we learn from these fields? in all of them you have to put in the initial effort. superlinear returns seem small at first. *at this rate,* you find yourself thinking, *i'll never get anywhere.* but because the reward curve rises so steeply at the far end, it's worth taking extraordinary measures to get there.

in the startup world, the name for this principle is "do things that don't scale." if you pay a ridiculous amount of attention to your tiny initial set of customers, ideally you'll kick off exponential growth by word of mouth. but this same principle applies to anything that grows exponentially. learning, for example. when you first start learning something, you feel lost. but it's worth making the initial effort to get a toehold, because the more you learn, the easier it will get.

there's another more subtle lesson in the list of fields with superlinear returns: not to equate work with a job. for most of the 20th century the two were identical for nearly everyone, and as a result we've inherited a custom that equates productivity with having a job. even now to most people the phrase "your work" means their job. but to a writer or artist or scientist it means whatever they're currently studying or creating. for someone like that, their work is something they carry with them from job to job, if they have jobs at all. it may be done for an employer, but it's part of their portfolio.

ause its adherents feel threatened by rapidly increasing variation in performance.

9 corollary: parents who pressure their kids to work on something prestigious, like medicine, even though they have no interest in it, will be hosing them even more than they have in the past.

10 the original version of this paragraph was the first draft of "how to do great work." as soon as i wrote it i realized it was a more important topic than superlinear returns, so i paused the present essay to expand this paragraph into its own. practically nothing remains of the original version, because after i finished "how to do great work" i rewrote it based on that.

11 before the industrial revolution, people who got rich usually did it like emperors: capturing some resource made them more powerful and enabled them to capture more. now it can be done like a scientist, by discovering or building something uniquely valuable. most people who get rich use a mix of the old and the new ways, but in the most advanced economies the ratio has shifted dramatically toward discovery just in the last half century.

12 it's not surprising that conventional-minded people would dislike inequality if independent-mindedness is one of the biggest drivers of it. but it's not simply that they don't want anyone to have what they can't. the conventional-minded literally can't imagine what it's like to have novel ideas. so the whole phenomenon of great variation in performance seems unnatural to them, and when they encounter it they assume it must be due to cheating or to some malign external influence.

thanks to trevor blackwell, patrick collison, tyler cowen, jessica livingston, harj taggar, and garry tan for reading drafts of this.

i grew up believing that taste is just a matter of personal preference. each person has things they like, but no one's preferences are any better than anyone else's. there is no such thing as *good* taste.

like a lot of things i grew up believing, this turns out to be false, and i'm going to try to explain why.

one problem with saying there's no such thing as good taste is that it also means there's no such thing as good art. if there were good art, then people who liked it would have better taste than people who didn't. so if you discard taste, you also have to discard the idea of art being good, and artists being good at making it.

it was pulling on that thread that unravelled my childhood faith in relativism. when you're trying to make things, taste becomes a practical matter. you have to decide what to do next. would it make the painting better if i changed that part? if there's no such thing as better, it doesn't matter what you do. in fact, it doesn't matter if you paint at all. you could just go out and buy a ready-made blank canvas. if there's no such thing as good, that would be just as great an achievement as the ceiling of the sistine chapel. less laborious, certainly, but if you can achieve the same level of performance with less effort, surely that's more impressive, not less.

yet that doesn't seem quite right, does it?

audience

i think the key to this puzzle is to remember that art has an audience. art has a purpose, which is to interest its audience. good art (like good anything) is art that achieves its purpose particularly well. the meaning of "interest" can vary. some works of art are meant to shock, and others to please; some are meant to jump out at you, and others to sit quietly in the background. but all art has to work on an audience, and—here's the critical point—members of the audience share things in common.

for example, nearly all humans find human faces engaging. it seems to be wired into us. babies can recognize faces practically from birth. in fact, faces seem to have co-evolved with our interest in them; the face is the body's billboard. so all other things being equal, a painting with faces in it will interest people more than one without. [1]

one reason it's easy to believe that taste is merely personal preference is that, if it isn't, how do you pick out the people with better taste? there are billions of people, each with their own opinion; on what grounds can you prefer one to another? [2]

but if audiences have a lot in common, you're not in a position of having to choose one out of a random set of individual biases, because the set isn't random. all humans find faces engaging—practically by definition: face recognition is in our dna. and so having a notion of good art, in the sense of art that does its job well, doesn't require you to pick out a few individuals and label their opinions as correct. no matter who you pick, they'll find faces engaging.

of course, space aliens probably wouldn't find human faces engaging. but there might be other things they shared in common with us. the most likely source of examples is math. i expect space aliens would agree with us most of the time about which of two proofs was better. erdos thought so. he called a maximally elegant proof one out of god's book, and presumably god's book is universal. [3]

once you start talking about audiences, you don't have to argue simply that there are or aren't standards of taste. instead tastes are a series of concentric rings, like ripples in a pond. there are some things that will appeal to you and your friends, others that will appeal to most people your age, others that will appeal to most humans, and perhaps others that would appeal to most sentient beings (whatever that means).

crowded party. the louvre might as well replace it with copy; no one would be able to tell. and yet the mona lisa is a small, dark painting. if you found people who'd never seen an image of it and sent them to a museum in which it was hanging among other paintings with a tag labelling it as a portrait by an unknown fifteenth century artist, most would walk by without giving it a second look.

for the average person, brand dominates all other factors in the judgement of art. seeing a painting they recognize from reproductions is so overwhelming that their response to it as a painting is drowned out.

and then of course there are the tricks people play on themselves. most adults looking at art worry that if they don't like what they're supposed to, they'll be thought uncultured. this doesn't just affect what they claim to like; they actually make themselves like things they're supposed to.

that's why you can't just take a vote. though appeal to people is a meaningful test, in practice you can't measure it, just as you can't find north using a compass with a magnet sitting next to it. there are sources of error so powerful that if you take a vote, all you're measuring is the error.

we can, however, approach our goal from another direction, by using ourselves as guinea pigs. you're human. if you want to know what the basic human reaction to a piece of art would be, you can at least approach that by getting rid of the sources of error in your own judgements.

for example, while anyone's reaction to a famous painting will be warped at first by its fame, there are ways to decrease its effects. one is to come back to the painting over and over. after a few days the fame wears off, and you can start to see it as a painting. another is to stand close. a painting familiar from reproductions looks more familiar from ten feet away; close in you see details that get lost in reproductions, and which you're therefore seeing for the first time.

there are two main kinds of error that get in the way of seeing a work of art: biases you bring from your own circumstances, and tricks played by the artist. tricks are straightforward to correct for. merely being aware of them usually prevents them from working. for example, when i was ten i used to be very impressed by airbrushed lettering that looked like shiny metal. but once you study how it's done, you see that it's a pretty cheesy trick—one of the sort that relies on pushing a few visual buttons really hard to temporarily overwhelm the viewer. it's like trying to convince someone by shouting at them.

the way not to be vulnerable to tricks is to explicitly seek out and catalog them. when you notice a whiff of dishonesty coming from some kind of art, stop and figure out what's going on. when someone is obviously pandering to an audience that's easily fooled, whether it's someone making shiny stuff to impress ten year olds, or someone making conspicuously avant-garde stuff to impress would-be intellectuals, learn how they do it. once you've seen enough examples of specific types of tricks, you start to become a connoisseur of trickery in general, just as professional magicians are.

what counts as a trick? roughly, it's something done with contempt for the audience. for example, the guys designing ferraris in the 1950s were probably designing cars that they themselves admired. whereas i suspect over at general motors the marketing people are telling the designers, "most people who buy suvs do it to seem manly, not to drive off-road. so don't worry about the suspension; just make that sucker as big and tough-looking as you can." [6]

i think with some effort you can make yourself nearly immune to tricks. it's harder to escape the influence of your own circumstances, but you can at least move in that direction. the way to do it is to travel widely, in both time and space. if you go and see all the different kinds of things people like in other cultures, and learn about all the different things people have liked in the past, you'll probably find it changes what you like. i doubt you could ever make yourself into a completely universal person, if only

"well," the professor replied, "we're interested in different questions now." he was a pretty nice guy, but at the time i couldn't help wishing i could send him back to fifteenth century florence to explain in person to leonardo & co. how we had moved beyond their early, limited concept of art. just imagine that conversation.

in fact, one of the reasons artists in fifteenth century florence made such great things was that they believed you could make great things. [10] they were intensely competitive and were always trying to outdo one another, like mathematicians or physicists today—maybe like anyone who has ever done anything really well.

the idea that you could make great things was not just a useful illusion. they were actually right. so the most important consequence of realizing there can be good art is that it frees artists to try to make it. to the ambitious kids arriving at art school this year hoping one day to make great things, i say: don't believe it when they tell you this is a naive and outdated ambition. there is such a thing as good art, and if you try to make it, there are people who will notice.

---

notes

1 this is not to say, of course, that good paintings must have faces in them, just that everyone's visual piano has that key on it. there are situations in which you want to avoid faces, precisely because they attract so much attention. but you can see how universally faces work by their prevalence in advertising.

2 the other reason it's easy to believe is that it makes people feel good. to a kid, this idea is crack. in every other respect they're constantly being told that they have a lot to learn. but in this they're perfect. their opinion carries the same weight as any adult's. you should probably question anything you believed as a kid that you'd want to believe this much.

3 it's conceivable that the elegance of proofs is quantifiable, in the sense that there may be some formal measure that turns out to coincide with mathematicians' judgements. perhaps it would be worth trying to make a formal language for proofs in which those considered more elegant consistently came out shorter (perhaps after being macroexpanded or compiled).

4 maybe it would be possible to make art that would appeal to space aliens, but i'm not going to get into that because (a) it's too hard to answer, and (b) i'm satisfied if i can establish that good art is a meaningful idea for human audiences.

5 if early abstract paintings seem more interesting than later ones, it may be because the first abstract painters were trained to paint from life, and their hands thus tended to make the kind of gestures you use in representing physical things. in effect they were saying "scaramara" instead of "uebfgbsb."

6 it's a bit more complicated, because sometimes artists unconsciously use tricks by imitating art that does.

7 i phrased this in terms of the taste of apples because if people can see the apples, they can be fooled. when i was a kid most apples were a variety called red delicious that had been bred to look appealing in stores, but which didn't taste very good.

8 to be fair, curators are in a difficult position. if they're dealing with recent art, they have to include things in shows that they think are bad. that's because the test for what gets included in shows is basically the market price, and for recent art that is largely determined by successful businessmen and their wives. so it's not always intellectual dishonesty that makes curators and dealers use neutral-sounding language.

9 what happens in practice is that everyone gets really good at *talking* about art. as the art itself gets more random, the effort that would have gone into the work goes instead into the intellectual sounding theory behind it. "my work represents an exploration of gender and sexuality in an urban context," etc. different people win at that game.

january 2006

to do something well you have to like it. that idea is not exactly novel. we've got it down to four words: "do what you love." but it's not enough just to tell people that. doing what you love is complicated.

the very idea is foreign to what most of us learn as kids. when i was a kid, it seemed as if work and fun were opposites by definition. life had two states: some of the time adults were making you do things, and that was called work; the rest of the time you could do what you wanted, and that was called playing. occasionally the things adults made you do were fun, just as, occasionally, playing wasn't — for example, if you fell and hurt yourself. but except for these few anomalous cases, work was pretty much defined as not-fun.

and it did not seem to be an accident. school, it was implied, was tedious *because* it was preparation for grownup work.

the world then was divided into two groups, grownups and kids. grownups, like some kind of cursed race, had to work. kids didn't, but they did have to go to school, which was a dilute version of work meant to prepare us for the real thing. much as we disliked school, the grownups all agreed that grownup work was worse, and that we had it easy.

teachers in particular all seemed to believe implicitly that work was not fun. which is not surprising: work wasn't fun for most of them. why did we have to memorize state capitals instead of playing dodgeball? for the same reason they had to watch over a bunch of kids instead of lying on a beach. you couldn't just do what you wanted.

i'm not saying we should let little kids do whatever they want. they may have to be made to work on certain things. but if we make kids work on dull stuff, it might be wise to tell them that tediousness is not the defining quality of work, and indeed that the reason they have to work on dull stuff now is so they can work on more interesting stuff later. [1]

once, when i was about 9 or 10, my father told me i could be whatever i wanted when i grew up, so long as i enjoyed it. i remember that precisely because it seemed so anomalous. it was like being told to use dry water. whatever i thought he meant, i didn't think he meant work could *literally* be fun — fun like playing. it took me years to grasp that.

jobs

by high school, the prospect of an actual job was on the horizon. adults would sometimes come to speak to us about their work, or we would go to see them at work. it was always understood that they enjoyed what they did. in retrospect i think one may have: the private jet pilot. but i don't think the bank manager really did.

the main reason they all acted as if they enjoyed their work was presumably the upper-middle class convention that you're supposed to. it would not merely be bad for your career to say that you despised your job, but a social faux-pas.

why is it conventional to pretend to like what you do? the first sentence of this essay explains that. if you have to like something to do it well, then the most successful people will all like what they do. that's where the upper-middle class tradition comes from. just as houses all over america are full of chairs that are, without the owners even knowing it, nth-degree imitations of chairs designed 250 years

unproductive pleasures pall eventually. after a while you get tired of lying on the beach. if you want to stay happy, you have to do something.

as a lower bound, you have to like your work more than any unproductive pleasure. you have to like what you do enough that the concept of "spare time" seems mistaken. which is not to say you have to spend all your time working. you can only work so much before you get tired and start to screw up. then you want to do something else — even something mindless. but you don't regard this time as the prize and the time you spend working as the pain you endure to earn it.

i put the lower bound there for practical reasons. if your work is not your favorite thing to do, you'll have terrible problems with procrastination. you'll have to force yourself to work, and when you resort to that the results are distinctly inferior.

to be happy i think you have to be doing something you not only enjoy, but admire. you have to be able to say, at the end, wow, that's pretty cool. this doesn't mean you have to make something. if you learn how to hang glide, or to speak a foreign language fluently, that will be enough to make you say, for a while at least, wow, that's pretty cool. what there has to be is a test.

so one thing that falls just short of the standard, i think, is reading books. except for some books in math and the hard sciences, there's no test of how well you've read a book, and that's why merely reading books doesn't quite feel like work. you have to do something with what you've read to feel productive.

i think the best test is one gino lee taught me: to try to do things that would make your friends say wow. but it probably wouldn't start to work properly till about age 22, because most people haven't had a big enough sample to pick friends from before then.

sirens

what you should not do, i think, is worry about the opinion of anyone beyond your friends. you shouldn't worry about prestige. prestige is the opinion of the rest of the world. when you can ask the opinions of people whose judgement you respect, what does it add to consider the opinions of people you don't even know? [4]

this is easy advice to give. it's hard to follow, especially when you're young. [5] prestige is like a powerful magnet that warps even your beliefs about what you enjoy. it causes you to work not on what you like, but what you'd like to like.

that's what leads people to try to write novels, for example. they like reading novels. they notice that people who write them win nobel prizes. what could be more wonderful, they think, than to be a novelist? but liking the idea of being a novelist is not enough; you have to like the actual work of novel-writing if you're going to be good at it; you have to like making up elaborate lies.

prestige is just fossilized inspiration. if you do anything well enough, you'll *make* it prestigious. plenty of things we now consider prestigious were anything but at first. jazz comes to mind — though almost any established art form would do. so just do what you like, and let prestige take care of itself.

prestige is especially dangerous to the ambitious. if you want to make ambitious people waste their time on errands, the way to do it is to bait the hook with prestige. that's the recipe for getting people to give talks, write forewords, serve on committees, be department heads, and so on. it might be a good rule simply to avoid any prestigious task. if it didn't suck, they wouldn't have had to make it prestigious.

when they're 12, and just glide along as if they were on railroad tracks. but this seems the exception. more often people who do great things have careers with the trajectory of a ping-pong ball. they go to school to study a, drop out and get a job doing b, and then become famous for c after taking it up on the side.

sometimes jumping from one sort of work to another is a sign of energy, and sometimes it's a sign of laziness. are you dropping out, or boldly carving a new path? you often can't tell yourself. plenty of people who will later do great things seem to be disappointments early on, when they're trying to find their niche.

is there some test you can use to keep yourself honest? one is to try to do a good job at whatever you're doing, even if you don't like it. then at least you'll know you're not using dissatisfaction as an excuse for being lazy. perhaps more importantly, you'll get into the habit of doing things well.

another test you can use is: always produce. for example, if you have a day job you don't take seriously because you plan to be a novelist, are you producing? are you writing pages of fiction, however bad? as long as you're producing, you'll know you're not merely using the hazy vision of the grand novel you plan to write one day as an opiate. the view of it will be obstructed by the all too palpably flawed one you're actually writing.

"always produce" is also a heuristic for finding the work you love. if you subject yourself to that constraint, it will automatically push you away from things you think you're supposed to work on, toward things you actually like. "always produce" will discover your life's work the way water, with the aid of gravity, finds the hole in your roof.

of course, figuring out what you like to work on doesn't mean you get to work on it. that's a separate question. and if you're ambitious you have to keep them separate: you have to make a conscious effort to keep your ideas about what you want from being contaminated by what seems possible. [6]

it's painful to keep them apart, because it's painful to observe the gap between them. so most people pre-emptively lower their expectations. for example, if you asked random people on the street if they'd like to be able to draw like leonardo, you'd find most would say something like "oh, i can't draw." this is more a statement of intention than fact; it means, i'm not going to try. because the fact is, if you took a random person off the street and somehow got them to work as hard as they possibly could at drawing for the next twenty years, they'd get surprisingly far. but it would require a great moral effort; it would mean staring failure in the eye every day for years. and so to protect themselves people say "i can't."

another related line you often hear is that not everyone can do work they love — that someone has to do the unpleasant jobs. really? how do you make them? in the us the only mechanism for forcing people to do unpleasant jobs is the draft, and that hasn't been invoked for over 30 years. all we can do is encourage people to do unpleasant work, with money and prestige.

if there's something people still won't do, it seems as if society just has to make do without. that's what happened with domestic servants. for millennia that was the canonical example of a job "someone had to do." and yet in the mid twentieth century servants practically disappeared in rich countries, and the rich have just had to do without.

so while there may be some things someone has to do, there's a good chance anyone saying that about any particular job is mistaken. most unpleasant jobs would either get automated or go undone if no one were willing to do them.

two routes

various types of work are like. at best you may have a couple internships, but not all jobs offer internships, and those that do don't teach you much more about the work than being a batboy teaches you about playing baseball.

in the design of lives, as in the design of most other things, you get better results if you use flexible media. so unless you're fairly sure what you want to do, your best bet may be to choose a type of work that could turn into either an organic or two-job career. that was probably part of the reason i chose computers. you can be a professor, or make a lot of money, or morph it into any number of other kinds of work.

it's also wise, early on, to seek jobs that let you do many different things, so you can learn faster what various kinds of work are like. conversely, the extreme version of the two-job route is dangerous because it teaches you so little about what you like. if you work hard at being a bond trader for ten years, thinking that you'll quit and write novels when you have enough money, what happens when you quit and then discover that you don't actually like writing novels?

most people would say, i'd take that problem. give me a million dollars and i'll figure out what to do. but it's harder than it looks. constraints give your life shape. remove them and most people have no idea what to do: look at what happens to those who win lotteries or inherit money. much as everyone thinks they want financial security, the happiest people are not those who have it, but those who like what they do. so a plan that promises freedom at the expense of knowing what to do with it may not be as good as it seems.

whichever route you take, expect a struggle. finding work you love is very difficult. most people fail. even if you succeed, it's rare to be free to work on what you want till your thirties or forties. but if you have the destination in sight you'll be more likely to arrive at it. if you know you can love work, you're in the home stretch, and if you know what work you love, you're practically there.

---

notes

1 currently we do the opposite: when we make kids do boring work, like arithmetic drills, instead of admitting frankly that it's boring, we try to disguise it with superficial decorations.

2 one father told me about a related phenomenon: he found himself concealing from his family how much he liked his work. when he wanted to go to work on a saturday, he found it easier to say that it was because he "had to" for some reason, rather than admitting he preferred to work than stay home with them.

3 something similar happens with suburbs. parents move to suburbs to raise their kids in a safe environment, but suburbs are so dull and artificial that by the time they're fifteen the kids are convinced the whole world is boring.

4 i'm not saying friends should be the only audience for your work. the more people you can help, the better. but friends should be your compass.

5 donald hall said young would-be poets were mistaken to be so obsessed with being published. but you can imagine what it would do for a 24 year old to get a poem published in *the new yorker*. now to people he meets at parties he's a real poet. actually he's no better or worse than he was before, but to a clueless audience like that, the approval of an official authority makes all the difference. so it's a harder problem than hall realizes. the reason the young care so much about prestige is that the people they want to impress are not very discerning.

6 this is isomorphic to the principle that you should prevent your beliefs about how things are from being contaminated by how you wish they were. most people let them mix pretty promiscuously. the continuing